

CAP-372: Processamento de Alto Desempenho (PAD)

Celso L. Mendes & Stephan Stephany

CAP /INPE

Email: celsoluizmendes@gmail.com stephan.stephany@gmail.com

Estrutura Planejada

- Aulas de 29/6 e 1/7/2026:
 - Hierarquia de Memória
 - i. Princípios de Memórias Caches
 - ii. Estruturas de Memória Principal (Livro – Cap.8)
 - iii. Memória Virtual (Livro – Cap.9)

Hierarquia de Memória

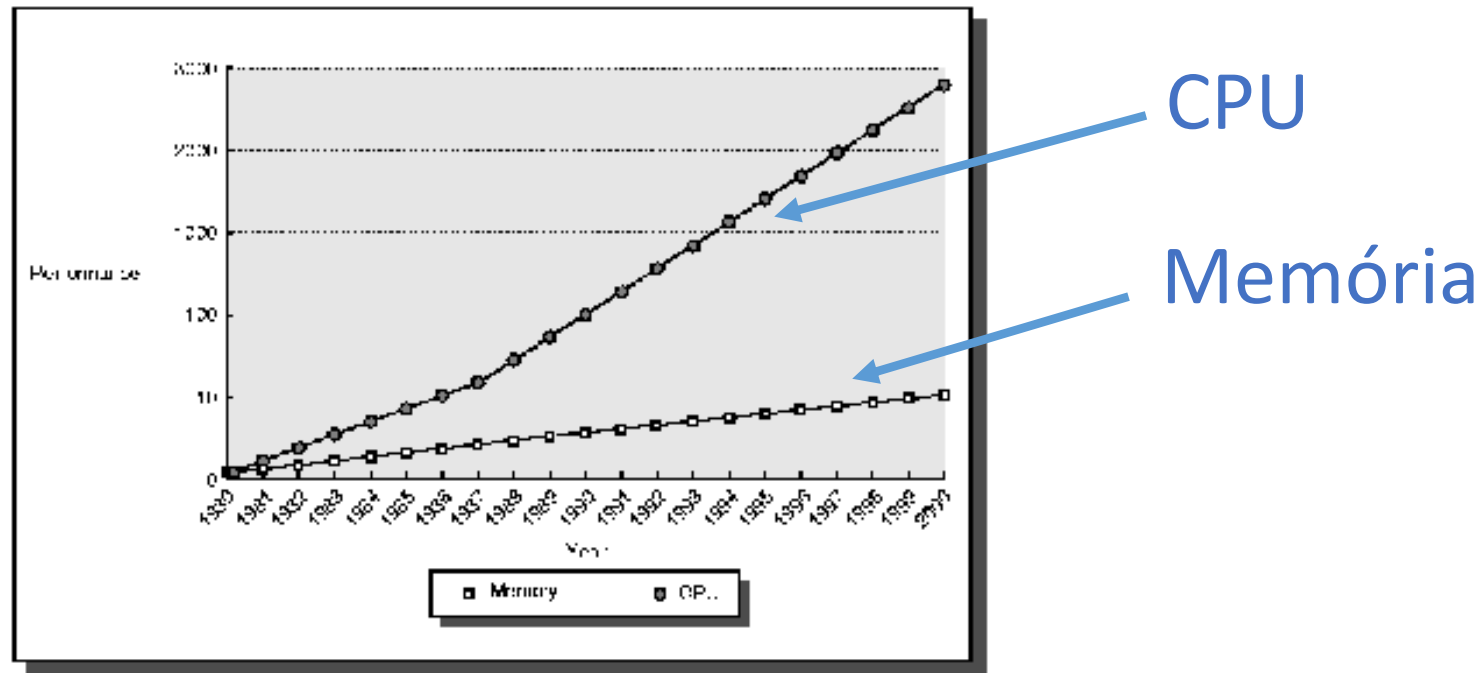
- Sistema de memória ideal:
 - Tamanho “ilimitado”, velocidade mais alta possível
- Solução economicamente viável:
 - Criar hierarquia de memória
 - Memórias de diversos tamanhos e velocidades, em níveis
 - Velocidade efetiva próxima à do nível mais alto
 - Tamanho efetivo é o do nível mais baixo
 - Cada nível contém um subconjunto dos dados do nível abaixo
 - Transferência de dados entre níveis é apoiada por hardware

Estrutura Planejada

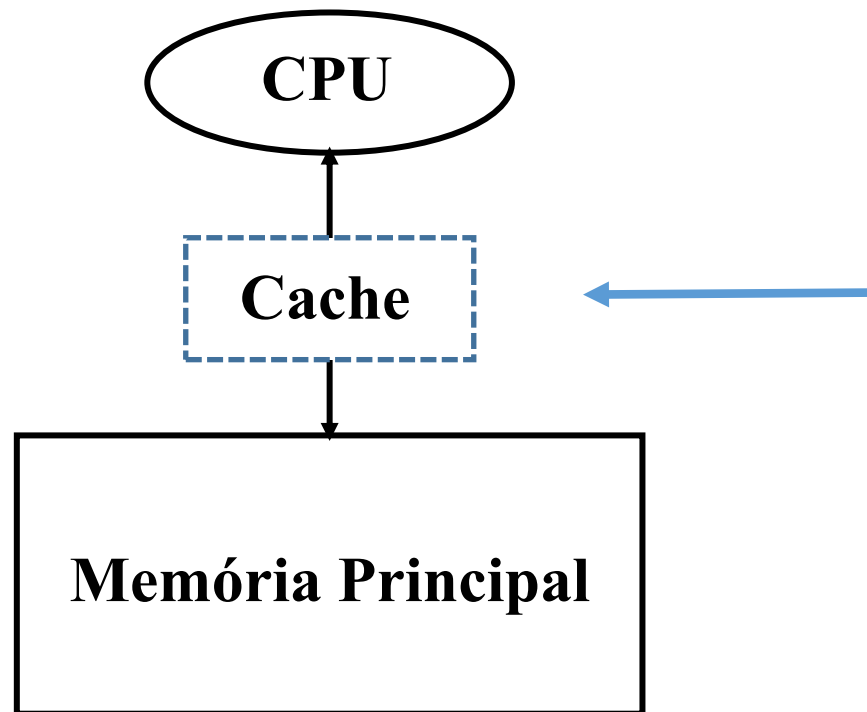
- Aulas de 29/6 e 1/7/2026:
 - Hierarquia de Memória
 - i. Princípios de Memórias Caches
 - ii. Estruturas de Memória Principal (Livro – Cap.8)
 - iii. Memória Virtual (Livro – Cap.9)

Motivação para Caches

- **Evolução de Desempenhos até 2000: CPU x Memória**
 - Memória: 7%/ano ; CPU: 35%-55%/ano



Motivação para Caches (cont.)

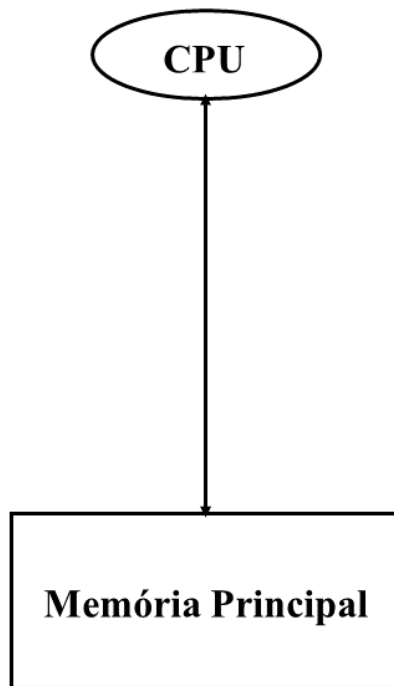


Novo
componente:
maior velocidade,
menor capacidade
que a mem. principal

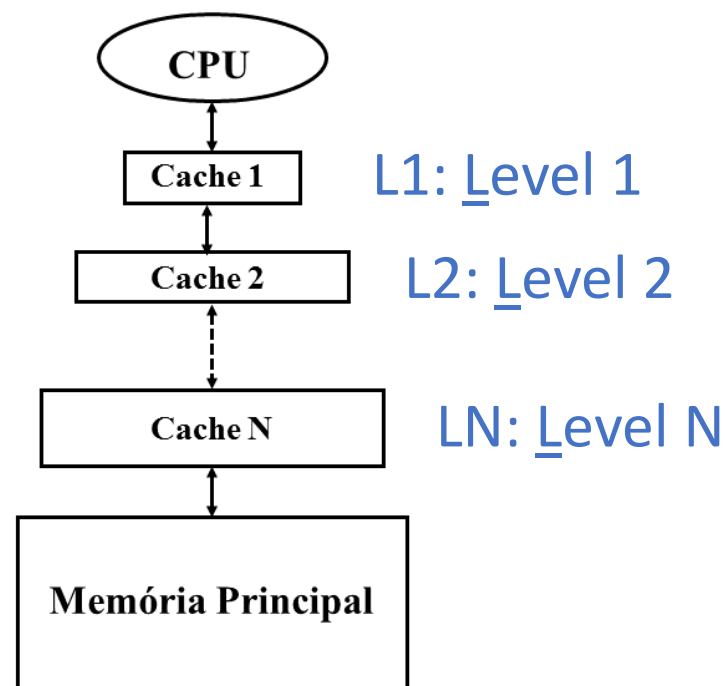
Hierarquia de Memória (cont.)

- Estrutura mais geral: múltiplos níveis de cache

Esquema Tradicional



Esquema com Hierarquia de Memória



L1: Level 1

L2: Level 2

LN: Level N

Velocidade

Capacidade

Hierarquia de Memória (cont.)

- Efetividade da cache: Princípios da *Localidade*
- Localidade Temporal
 - Se um dado é acessado, deverá ser acessado novamente em futuro próximo
- Localidade Espacial
 - Se um dado é acessado, dados próximos em memória deverão ser acessados em futuro próximo
 - Razão para estruturar a cache em blocos (linhas)
- Ambos os princípios são observados na prática
 - Caches existem em todos os sistemas atuais

Hierarquia de Memória (cont.)

- **Processadores modernos**
 - Um ou mais níveis de cache são internos ao chip da CPU
 - Velocidade interna bem maior que a externa
 - Pode haver caches separadas para dados e instruções
 - Em geral, caches de instruções são menores
 - Otimização: instruções específicas de pré-carga (*prefetch*)
- **Possível problema: consistência dos dados em cache**
 - Exemplo: periféricos ligados diretamente à memória podem atualizar a memória em posições que estão na cache
 - Solução: hardware de monitoramento, invalidação dos dados em cache no caso de atualizações da memória

Estrutura Básica de Caches

- **Estrutura popular de armazenamento dos dados:**
 - Unidade básica: bloco de dados (conjunto contíguo de bytes)
 - Algumas vezes também chamado de linha
 - Endereço do bloco = subconjunto dos bits de endereço dos bytes
 - Transferências entre níveis sempre ocorrem por blocos
- **Política de controle em cada nível:**
 - Onde colocar um certo bloco?
 - Como saber se um certo bloco está presente na cache?
 - Qual bloco deve ser retirado ao se trazer um novo bloco?
 - O que acontece durante um *write*?

Estrutura Planejada

- Aulas de 29/6 e 1/7/2026:
 - Hierarquia de Memória
 - i. Princípios de Memórias Caches
 - ii. Estruturas de Memória Principal (Livro – Cap.8)
 - iii. Memória Virtual (Livro – Cap.9)

Estruturas de Memória Principal

Nomenclatura usual:

- **Latência:** tempo exigido para acessar *um* dado (em segundos)
- **Tempo de acesso:** tempo mínimo entre colocação do endereço e chegada dos dados
- **Tempo de ciclo:** tempo mínimo entre dois acessos consecutivos
- **Largura de Faixa** (ou Largura de **Banda**): inverso do tempo médio por dado, para acesso a um *bloco* de dados (em bytes/segundo)
- **Memórias Estáticas:** ênfase na velocidade
- **Memórias Dinâmicas:** ênfase na capacidade

Desempenho de Memória Principal

- **Evolução Típica de Memórias Dinâmicas:**
 - Aumento típico de capacidade de $4\times$ a cada ~ 3 anos
 - Suficiente para acompanhar demandas de programas
 - Aumento típico de velocidade de $\sim 7\%$ ao ano
 - Insuficiente para acompanhar demandas da CPU
 - Alternativas:
 - Utilização de caches (porém, custo é bem mais alto)
 - Implementadas com memória estática (bem menores)
 - Utilização de organizações de memória mais eficazes
 - Ex: interleaving, com vários bancos de memória

Memória com Interleaving

- **Idéia:**
 - Dividir a memória em bancos distintos
 - Dirigir acessos consecutivos para blocos separados
 - Acessar blocos “em paralelo” (ao mesmo tempo)
- **Objetivo Final:** um ciclo/acesso (desde que acessos consecutivos sejam a bancos distintos)
- **Estratégia Típica:** mapear endereços sequenciais para bancos distintos – otimização via acessos em paralelo
 - Endereços sequenciais = bloco/linha de cache

Memória com Interleaving (cont.)

Interleaving com 4 bancos de memória:

Address	Bank 0	Address	Bank 1	Address	Bank 2	Address	Bank 3
0		1		2		3	
4		5		6		7	
8		9		10		11	
12		13		14		15	

Mapeamento: endereço $\rightarrow$ {banco, offset}

- banco = endereço MOD núm.bancos
- offset = endereço / núm.bancos

Melhor sequência de acessos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10...

Todos os bancos serão acessados

Pior sequência de acessos: 0, 4, 8, 12, 16, ...

Todos os acessos vão para um único banco!

Memória com Interleaving (cont.)

- Mesmo com interleaving, pode ainda haver conflitos!

Exemplo: (supondo 128 bancos de memória)

```
int x[256][512];  
for (j=0; j<512; j++)  
    for (i=0; i<256; i++) x[i][j] = 2 * x[i][j];
```

→ Como 512 é múltiplo de 128, $x[i][j]$ e $x[i+1][j]$ estarão sempre no mesmo banco!

Soluções Possíveis:

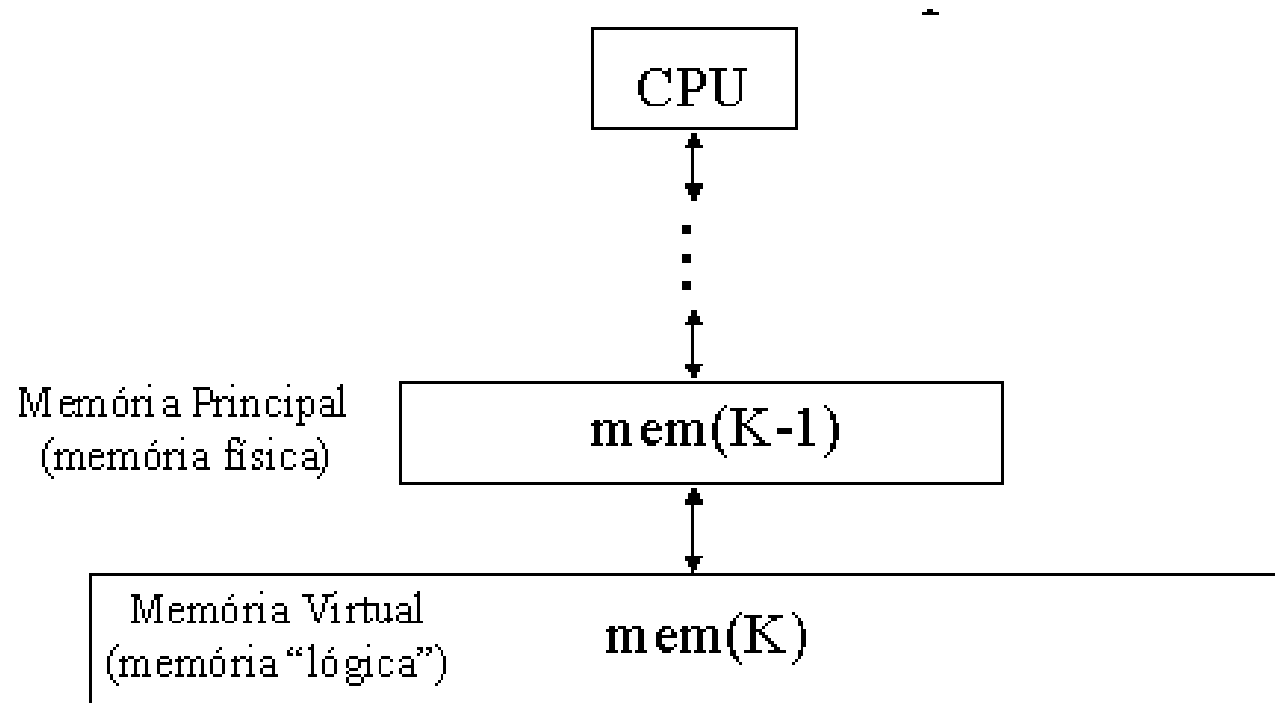
- Mudar tamanho do array x (ex: inserir elementos adicionais, sem uso)
- Fazer núm.bancos = número ímpar/primo (pouco usada)

Estrutura Planejada

- Aulas de 29/6 e 1/7/2026:
 - Hierarquia de Memória
 - i. Princípios de Memórias Caches
 - ii. Estruturas de Memória Principal (Livro – Cap.8)
 - iii. Memória Virtual (Livro – Cap.9)

Memória Virtual

- **Conceito:** Último nível da hierarquia de memória



Memória Virtual (cont.)

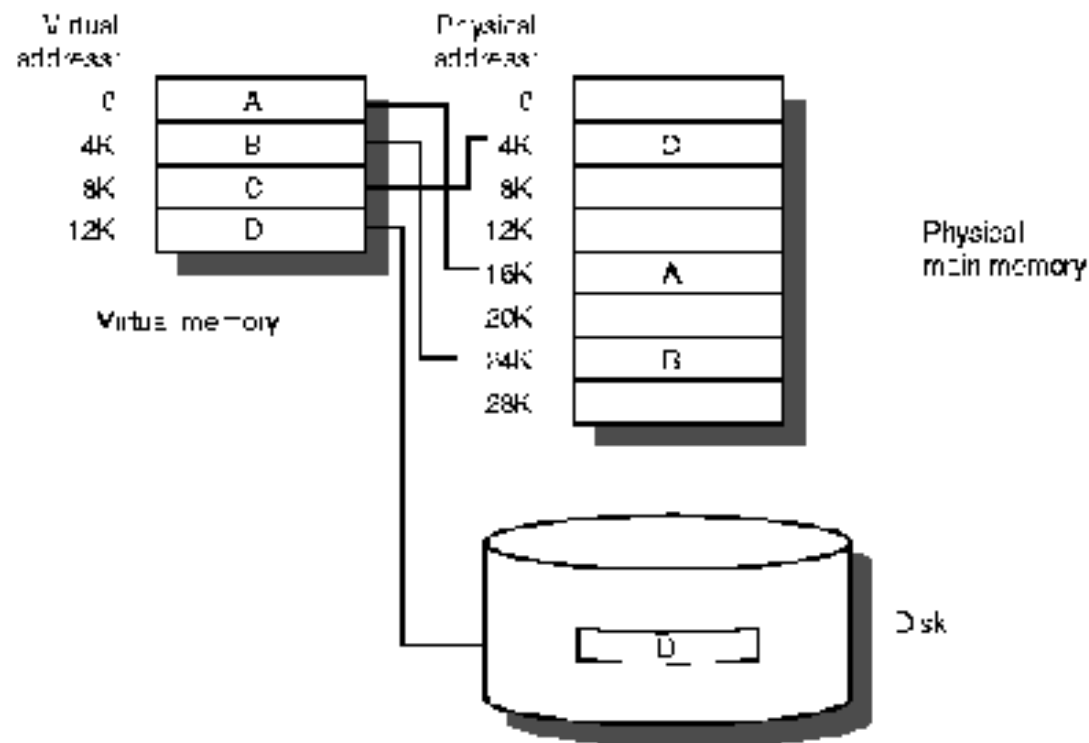
- **Objetivo:** Dar aos programas a “ilusão” de uma memória de tamanho *infinito* (ou muito grande – bem maior que a memória física real)
- **Esquema de Controle:** mapeamento automático entre endereços virtuais (do programa) e físicos (da memória)
- **Vantagens:**
 - Tamanho do programa independe do tamanho da memória
 - Memória real pode ser compartilhada por vários programas
 - Não é preciso carregar todo o programa para iniciar sua execução
 - Programa pode ser executado em qualquer região da memória física

Memória Virtual (cont.)

- **Implementação:**
 - Dividir espaço de endereçamento em páginas
 - Carregar cada página para a memória física, conforme necessário
 - CPU processa o programa com endereços *virtuais*
 - Memória é acessada com endereços *reais*
 - Tradução de endereços *virtuais* para *reais* feita por hardware + software
 - Páginas que não estão sendo utilizadas ficam em disco

Memória Virtual (cont.)

- Exemplo de execução com memória virtual:



Memória Virtual (cont.)

Comparação: Cache × Memória Virtual

- Substituição de Blocos/Páginas:
 - Cache: controle por hardware; unidade=bloco
 - Mem.Virtual: controle por software (Sist. Operacional); unidade=página
- Tamanho Total:
 - Cache: livre escolha
 - Memória Virtual: definido pelo campo de endereço da CPU
- Localização:
 - Cache: módulo de memória exclusivo
 - Memória Virtual: em disco, junto com o sistema de arquivos

Memória Virtual (cont.)

Controle de Sistemas de Memória Virtual

- Onde colocar uma nova página na memória principal?
 - Qualquer local na memória (Objetivo: minimizar a taxa de erro - *miss*)
- Como descobrir se uma certa página está na memória?
 - Estrutura de dados: Tabela de Páginas
 - Dado o número da página, fornece seu endereço inicial
 - Offset dentro da página pode ser usado diretamente no endereço físico

Exemplo: Endereço virtual de 28 bits, páginas de 4K bytes

Endereço Virtual = 16 bits (página) + 12 bits (offset)

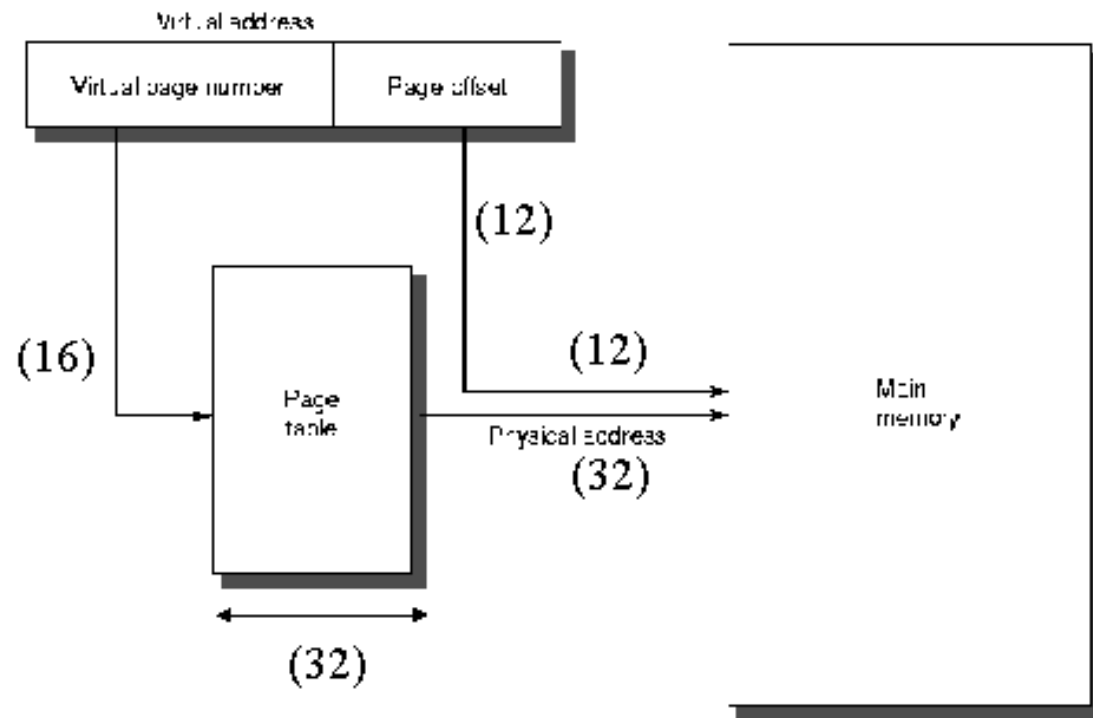
Supondo 4 bytes por entrada na Tab.Páginas:

Tamanho total da Tab.Páginas = $2^{16} * 4 \text{ bytes} = 256 \text{ Kbytes}$

Memória Virtual (cont.)

Mapeamento Endereço Virtual → Endereço Físico:

- Pode ser acelerado por hardware (TLB: Translation Lookaside Buffer)



Memória Virtual (cont.)

- **Observações:**

- Tab.Páginas e TLB são controlados pelo Sist.Operacional
- Tab.Páginas tem tamanho fixo (definido pelo tamanho do endereço virtual)
- Memória real pode ter qualquer tamanho; basta informar ao Sist. Operac.
- Tab.Páginas deve ter bit informando se a página está presente na memória
- Em cada acesso à memória principal:
 - Se a página estiver presente (*hit*): dado é acessado de imediato
 - Se a página não estiver presente (*miss: page-fault*): página deve ser carregada do disco para a memória principal pelo Sist.Oper. (operação extremamente demorada)
- Algumas das técnicas de rearranjo de programas usadas para minimizar cache-misses podem ser usadas também para minimizar page-faults (observando os respectivos tamanhos dos blocos - linhas × páginas)